L-Int: LL-Parser mit Recursive Descent

Carsten Gips (HSBI)

Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

LL-Parser mit Recursive Descent: Ziel

- Aus Tokens einen AST bauen:

```
Expr parse(List<Token> tokens);
```

- Basis konkrete Syntax (Grammatik):

```
eform ::= '(' '+' eform eform+ ')' | '(' '-' eform eform+ ')' | INT
INT   ::= [0-9]+
```

- Zielformat AST (abstrakte Syntax):

```
Expr  = IntLiteral(Int) | AddExpr(Expr, Expr) | SubExpr(Expr, Expr)
```

Hinweis: Schnittstelle, AST vs. Parse-Tree

Grundidee LL-Parser mit Recursive Descent

```
r ::= X s
```

```
Expr parseR() {
    match(X);
    return parseS();
}
```

Alternative Subregeln

```
    a | b | c
```

```
return switch (peek()) {
    case PredictingA -> parseA();
    case PredictingB -> parseB();
    case PredictingC -> parseC();
    default -> throw new RuntimeException("expected expression");
};
```

Grundstruktur eines RD-Parsers

```
public class LispyParserLint {
    protected List<Token> tokens;
    protected int current;

    public Expr parse(List<Token> tokenList) {
        tokens = tokenList;  current = 0;

        Expr expr = parseEform();
        if (!isAtEnd()) throw new RuntimeException("unexpected token");

        return expr;
    }

    // Arbeitsmethoden: parseEform(), parseAtom(), parseAddExpr(), parseSubExpr()
    // Hilfsmethoden:   isAtEnd(), peek(), advance(), match(), check() ...
}
```

Partielle Beispielimplementierung RD-Parser für L-Int

```
protected Expr parseEform() {
    if (check(LPAREN)) {
        return switch (peek(1).kind()) {
            case PLUS -> parseAddExpr();
            default -> throw new RuntimeException("unexpected token");
        };
    }

    return parseAtom();
}


protected Expr parseAtom() {
    Token t = match(INT);
    return new IntLiteral(Integer.parseInt(t.lexeme()));
}
```

Details Parser für L-Int: `parseAddExpr()`

```
'(' '+' eform eform+ ')'
```

```
protected Expr parseAddExpr() {
    match(LPAREN);
    match(PLUS);

    List<Expr> ops = new ArrayList<>();
    ops.add(parseEform());
    while (!check(RPAREN)) ops.add(parseEform());

    match(RPAREN);

    return ops.stream().reduce(AddExpr::new).orElseThrow();
}
```

Hinweis: Desugaring

Hilfsfunktionen im Parser: Tokenhandling

```
protected boolean isAtEnd() { return peek().kind() == EOF; }

protected Token peek() { return peek(0); }
protected Token peek(int la) {
    if (la < 0 || current + la >= tokens.size()) return tokens.getLast();  // EOF
    return tokens.get(current + la);
}

protected Token advance() {
    return isAtEnd() ? tokens.getLast() : tokens.get(current++);
}

protected boolean check(TokenKind... expected) {
    return Arrays.stream(expected).anyMatch(e -> e == peek().kind());
}
protected Token match(TokenKind... expected) {
    if (check(expected)) return advance();
    throw new RuntimeException("unexpected token: " + peek());
}
```

Auflösen von (links-) rekursiven Grammatiken

```
expr ::= expr '+' expr | INT
```

Auflösen von (links-) rekursiven Grammatiken

```
expr ::= expr '+' expr | INT
```

```
expr ::= expr '+' INT | INT
```

Auflösen von (links-) rekursiven Grammatiken

```
expr ::= expr '+' expr | INT
```

```
expr ::= expr '+' INT | INT
```

```
ruleA ::= ruleA a1 | ... | ruleA am | b1 | ... | bn |
```

```
ruleA  ::= b1 ruleAA | ... | bn ruleAA
ruleAA ::= a1 ruleAA | ... | am ruleAA | eps
```

Auflösen von (links-) rekursiven Grammatiken

```
expr ::= expr '+' expr | INT
```

```
expr ::= expr '+' INT | INT
```

```
ruleA ::= ruleA a1 | ... | ruleA am | b1 | ... | bn |
```

```
ruleA  ::= b1 ruleAA | ... | bn ruleAA
ruleAA ::= a1 ruleAA | ... | am ruleAA | eps
```

```
expr  ::= INT expr'
expr' ::= '+' INT expr' | eps
```

```
expr  ::= INT ('+' INT)*
```

Lispy-Pipeline komplett

```
String source = "(- (+ 1 2) 3)";

List<Token> tokens = new LispyLexerLint().scan(source);
Expr ast = new LispyParserLint().parse(tokens);

Value result = new InterpreterLint().eval(ast);
// result: IntVal(0)
```

Wrap-up

Implementierung als RD-Parser:

- Regeln (Nichtterminale) werden zu Parsermethoden
- Token (Terminals) mit `match()` prüfen (und "konsumieren")
- Auswahl Alternativen über nächste(s) Token(s) (Lookahead)
- Beachtung und Auflösung von Linksrekursion



Unless otherwise noted, this work is licensed under CC BY-SA 4.0.